

Guard-hook §6.T.3 force-push FALSE POSITIVE — root cause + proposed fix

Revision: 1 **Last modified:** 2026-07-09T23:59:00Z **Classification:** investigation finding (§11.4.102 systematic-debug). The fix targets a constitution/ file inherited by reference (§11.4.109) and MUST land via the §11.4.26 constitution-submodule workflow — NOT applied here. **Authority:** consumer-side forensic record (§11.4.35). Anti-bluff §11.4 — every claim below is a reproduced grep against the real hook, not a guess (§11.4.6).

1. Symptom

constitution/scripts/hooks/guard-forbidden-commands.sh (the §11.4.109 PreToolUse guard, wired in .claude/settings.json) BLOCKED a legitimate Bash command as “§6.T.3 force-push” during this session, although the command performed NO force-push (it ran git fetch origin main, git rev-parse, tail on a push log, export_docs.sh, and commit_all.sh --paths ... -m ... — commit_all.sh pushes fast-forward + detached, never --force). Splitting the command so the commit_all.sh call ran alone passed cleanly — proving the block was spurious.

2. Root cause (matcher lines 178–194)

- **OUTER (line 182):** if [["\$COMMAND" =~ git([[:space:]]+[^[:space:]]+)*([[:space:]]+push)] — the middle ([[:space:]]+[^[:space:]]+)* is **unbounded**, and [[:space:]] matches **newlines**, so the pattern spans from the FIRST git token to ANY whitespace-preceded push anywhere later in a multi-line command. It never verifies git and push are the same invocation.
- **INNER (lines 183–185):** scans the ENTIRE command for any --force / -f / --force-with-lease token — including benign non-git

flags (podman ... --force, grep -f, rm -f, cp -f, tar -f), or those words inside a quoted -m commit message.

- Both OUTER + INNER matched → block "\$6.T.3 force-push" (line 186).

In the blocked command: git fetch origin main supplied git; the echo "... T3 push landed ..." strings supplied a space-preceded push; a benign -f/--force token elsewhere completed INNER.

3. False-positive class (over-broad)

Any single Bash command containing simultaneously **(a)** a git <anything> token, **(b)** a whitespace-preceded literal push anywhere after it (echo text, a -m message mentioning git push, a push_all.sh call), and **(c)** any -f/--force/--force-with-lease token anywhere (including benign non-git flags) is falsely blocked — even with no git push --force present. This blocks routine commit_all.sh one-liners whose preamble/message happens to contain “push” plus a -f token. It is **fail-closed** (over-blocks; never lets a real force-push through), so the live risk is friction, not data loss.

Reproduced through the real hook (echo JSON | bash guard-forbidden-commands.sh):

Command shape	Verdict
git fetch + echo "T3 push ..." + podman rm --force	BLOCKED \$6.T.3 (EXIT 2)
same --force, preamble removed (grep -f ...; commit_all.sh)	ALLOW (EXIT 0)
git push --force origin main	BLOCKED (correct)

4. Proposed fix (PROPOSAL — must go through §11.4.26 + the ≥20-case hook test suite)

Make OUTER match a real git push invocation (only option-shaped tokens between git and push):

```
GIT_PUSH_RE='(^|[[[:space:]]]|;|\||&)git([[[:space:]]|+  
[[[:space:]]*])*[[[:space:]]|+push([[[:space:]]|$)'
```

Validated old-vs-proposed: correctly stops blocking the benign cases while still catching `git push --force / -f / --force-with-lease`.

Honest boundary (§11.4.6): this minimal regex introduces a false-NEGATIVE — it MISSES the `git -c key=val push --force global-option-with-value` form (the value token `key=val` breaks the option-only middle). In a force-push guard a false-negative is a SAFETY REGRESSION (§11.4.113 / §9.2). Therefore the minimal regex is NOT sufficient on its own. The correct fix is the fuller one:

Fuller fix: split `$COMMAND` on `; / && / || / | / newlines` into simple commands; flag only a simple command whose first word (after env assignments) is `git` and whose args contain `push` AND a force flag; scope the INNER `-f/--force` scan to that `git push` segment (so `git push origin main && grep -f x` no longer trips on the `grep -f`, and `git -c k=v push --force` is still caught).

5. Why it is NOT fixed in this session

The hook is inherited by reference (§11.4.109 — NEVER copied). Editing it is a constitution-submodule change requiring the §11.4.26 workflow (`fetch` → `edit` → `validate` → multi-upstream push to all constitution remotes) with the ≥20-case hermetic hook test suite updated to cover: real force-pushes still blocked (incl. `git -c k=v push --force`), the benign combos now allowed, and every other blocked class (`sudo`, `host-power`, `host-direct emulator/adb`) unaffected. Risk asymmetry (§11.4.101): the current bug is low-harm fail-closed friction; a wrong fix is high-harm fail-open (unrecoverable force-push slipping past §11.4.113). Deferred to the constitution workflow with operator awareness rather than rushed.

Sources verified

- `constitution/scripts/hooks/guard-forbidden-commands.sh` lines 178–194 (read 2026-07-09).
- Reproductions run against the real hook; fixtures in the session scratchpad only (not committed, §11.4.128 raw-evidence).